Computer Science Department

הטכניון
מכון טכנולוגי לישראל

LAB OF COMPUTER COMMUNICATION & NETWORKING
Networking Education, Research and Innovation
Computer Science Department, Technion - Israel Institute of Technology

Load Balancer Project

Project in Computer Networks - 236340 - Winter 2026

Team: Alaa Abo Mokh, Adan Abo Mokh, Mayyas Iktilat

Advisors: Eran Tavor

Abstract

This project examines the behavior of different load balancing algorithms in a controlled network environment. The main goal is to compare **IP Hash**, **Least Connections**, and an improved variant called **Least Connections — Second Tie** under several server and traffic conditions.

The project was implemented using a Mininet-based topology with multiple clients, a load balancer, and two backend servers. We tested the algorithms with symmetric servers, heterogeneous servers, different request gaps, and different mappings between clients and servers. The main metric used for comparison was **Total Completion Time (TCT)**, which represents the time required to complete all client requests in an experiment run.

The results show that the performance of a load balancing algorithm is not determined only by whether it is static or dynamic. It also depends on server speed, client distribution, request timing, failed packets, and implementation details such as tie-breaking. In particular, we found that the tie-breaking rule in Least Connections can significantly affect performance when the servers are not identical.

Table of Contents

1. Introduction
2. Background and Definitions
3. Project Topology
4. Project Stages
5. Experiments and Results
6. Main Conclusions
7. Final Summary
8. References

---

1. Introduction

Load balancing is an important technique in computer networks and distributed systems. When many clients send requests to the same service, a single server may become overloaded. A load balancer solves this problem by distributing the incoming requests among several backend servers. The decision of where to send each request is made according to a load balancing algorithm.

In this project, we focused on comparing two common load balancing approaches: **IP Hash** and **Least Connections**. IP Hash is a static algorithm that maps each client to a server based on the client IP address. Least Connections is a dynamic algorithm that chooses the server with the fewest active connections at the time of the request.

At the beginning of the project, we expected Least Connections to perform better in most cases because it reacts to the current state of the system. However, the experiments showed that the behavior is more complex. In some scenarios, Least Connections performed better as expected. In other scenarios, IP Hash was competitive or even better because the IP mapping happened to send more requests to the faster server.

During the experiments, we also found that the implementation of Least Connections had an important detail: when there was a tie between servers, the algorithm always selected the first server in the code. This behavior affected the results, especially when the servers had different speeds. To study this effect, we added a new algorithm called **Least Connections - Second Tie**, which selects the second server when there is a tie.

The rest of this report presents the background concepts, the experimental environment, the topology, the algorithms, the experiments, and the conclusions we reached from the results.

---

2. Background and Definitions

2.1 Load Balancer

A **load balancer** is a networking component that distributes incoming client requests among several backend servers. Instead of sending all requests to one server, the load balancer decides which server should handle each request.

The purpose of a load balancer is to improve:

- **Performance** — by reducing overload on a single server.
- **Availability** — by allowing the service to continue working even if one server is busy or unavailable.
- **Scalability** — by making it easier to add more servers when the system grows.
- **Resource utilization** — by using the available servers more efficiently.

In our project, the load balancer receives requests from clients and forwards them to one of two backend servers.

2.2 Backend Server

A **backend server** is a server that actually processes the client request. In our system, the clients do not directly communicate with the backend servers. Instead, they send requests to the load balancer, and the load balancer forwards each request to one of the servers.

We used two backend servers in the topology:

- **h3**
- **h4**

In some experiments, the servers were identical. In other experiments, one server was faster and the other was slower.

2.3 Client

A **client** is a host that sends requests to the service. In our experiments, several clients sent requests to the load balancer. Each request represented a type of work that the server had to process.

The request types used in the project were:

- **M** — Music request
- **V** — Video request
- **P** — Photo request

Each request type had a different processing time on the server.

2.4 Total Completion Time (TCT)

Total Completion Time (TCT) is the main metric we used to compare the algorithms. It measures the total time required to finish all requests in one experiment run.

A lower TCT means that the workload finished faster. Therefore, when comparing algorithms, the algorithm with the lower average TCT is considered better for that specific scenario.

We also looked at the **standard deviation** of TCT. The standard deviation helps us understand how stable the results are across different runs. A smaller standard deviation means the algorithm behaves more consistently.

2.5 Static Load Balancing Algorithm

A **static load balancing algorithm** makes routing decisions without looking at the current state of the servers. It does not know how many active connections each server currently has, and it does not react dynamically to changes in the system.

In our project, **IP Hash** is the static algorithm.

2.6 Dynamic Load Balancing Algorithm

A **dynamic load balancing algorithm** makes routing decisions according to the current state of the system. For example, it may check how many active connections each server has and choose the server with the smallest number.

In our project, **Least Connections** is the dynamic algorithm.

2.7 IP Hash Algorithm

The **IP Hash** algorithm maps each client IP address to a specific backend server. The same client is sent to the same server every time.

The main advantage of IP Hash is that it provides **consistent routing**. This can be useful in systems where we want the same client to continue using the same server, for example when session persistence is needed.

However, IP Hash does not check the current server load. This means that it can create an imbalance if many clients are mapped to the same server, especially if that server is slower.

In our project, the IP Hash mapping was sensitive to the client IP addresses. In some cases, it mapped more traffic to the slow server, which made IP Hash perform worse. In other cases, it mapped more traffic to the fast server, and then IP Hash performed surprisingly well.

2.8 Least Connections Algorithm

The **Least Connections** algorithm chooses the server with the fewest active connections. The idea is that a server with fewer active connections is probably less busy and can handle a new request faster.

This algorithm is dynamic because it reacts to the current state of the system.

In our first implementation, when both servers had the same number of active connections, the algorithm always chose the first server in the code. Later, we found that this tie-breaking decision had a large effect on the results.

2.9 Least Connections — Second Tie

After analyzing the behavior of Least Connections, we added a new version called **Least Connections — Second Tie**.

This version behaves like Least Connections, but when there is a tie between the servers, it chooses the second server instead of always choosing the first one.

This change was important because in some scenarios the second server was the faster server. Choosing it during ties improved the performance significantly.

This showed us that even a small implementation detail, such as tie-breaking, can strongly affect the final results.

2.10 Gap Between Requests

The **gap** is the time interval between client requests. A smaller gap means that requests arrive quickly and the system is under higher load. A larger gap means that requests are more spread out, so the system has more time to finish previous requests before receiving new ones.

We changed the gap values during the project to check whether the gap was the reason for the small differences between the algorithms. However, we found that the gap was not the only factor. The IP mapping, failed packets, and tie-breaking behavior were also very important.

2.11 Symmetric and Heterogeneous Servers

Symmetric servers are servers with identical processing times. In this case, both servers are equally fast.

Heterogeneous servers are servers with different processing times. One server may be faster while the other is slower.

This difference was central to our project because the algorithms behaved differently depending on whether the servers were identical or not.

2.12 Fast Server and Slow Server

In the heterogeneous experiments, we used one fast server and one slow server.

Example of a fast server:

M = 0.10 seconds

V = 0.30 seconds

P = 0.05 seconds

Example of a slow server:

M = 0.30 seconds

V = 0.80 seconds

P = 0.20 seconds

In the extreme heterogeneous scenario, the slow server became even slower:

M = 0.50 seconds

V = 1.20 seconds

P = 0.30 seconds

These service times helped us create different levels of imbalance between the servers.

2.13 Mininet

Mininet is a network emulator that allows us to create virtual hosts, switches, links, and controllers on one machine. It uses the real Linux networking stack, so it gives a realistic environment for testing network behavior.

We used Mininet to build the topology of our project and run the experiments in a controlled way.

---

3. Project Topology

The topology used in the project was simple:

Clients → Load Balancer → Servers

The three clients were connected to the load balancer. The load balancer was connected to two servers:

- **h3**
- **h4**

The load balancer received all client requests and decided whether to forward each request to h3 or h4.

This topology allowed us to compare the algorithms under the same network structure while changing only the server speeds, request gap, and algorithm logic.

---

4. Project Stages

The work was done in several stages:

1. **Environment setup**
   We installed and prepared the required tools, including Mininet and Ryu.
2. **Topology setup**
   We built the network topology with clients, a load balancer, and two backend servers.
3. **Traffic simulation**
   We created client traffic using different request types.
4. **Load balancer implementation**
   We implemented IP Hash and Least Connections.
5. **Performance measurement**
   We measured Total Completion Time for each experiment.
6. **Repeated runs**
   We repeated the experiments many times to get more stable results.
7. **Results visualization**
   We created graphs to compare the algorithms and understand their behavior.
8. **Algorithm improvement**
   After observing that the tie-breaking policy in Least Connections had a strong effect on the results, we added the Least Connections — Second Tie algorithm.

---

5. Experiments and Results

5.1 Initial Expectation

Before running the experiments, we expected Least Connections to perform better than IP Hash. The reason is that Least Connections uses the current number of active connections, while IP Hash only uses the client IP address.

We expected IP Hash to be useful mainly for consistency, but not necessarily for performance. We also expected Least Connections to be better when the traffic was not evenly distributed.

However, the experiments showed that the full picture is more complicated.

5.2 Baseline Experiment — Symmetric Servers

We started with a baseline experiment where the servers were symmetric, meaning both servers had the same service times:

M = 0.15 seconds

V = 0.40 seconds

P = 0.08 seconds

The gap range was:

GAP_MIN = 0.05

GAP_MAX = 0.2

The results were:

Average IP Hash TCT = 119.891 seconds

Average Least Connections TCT = 116.662 seconds

Least Connections was slightly better, but the difference was not very large. This was a little surprising because we expected Least Connections to clearly outperform IP Hash.

5.3 Heterogeneous Setup — One Slow Server

Next, we changed the setup so that one server was slower and one server was faster.

Slow server:

M = 0.30 seconds

V = 0.80 seconds

P = 0.20 seconds

Fast server:

M = 0.10 seconds

V = 0.30 seconds

P = 0.05 seconds

The results were:

Average IP Hash TCT = 204.078 seconds

Average Least Connections TCT = 164.542 seconds

In this experiment, Least Connections was clearly better. This result matched our expectation.

The reason is that IP Hash may send many requests to the slow server depending on the client IP mapping. Since IP Hash does not react to the current server load or speed, it can keep sending traffic to a server that becomes a bottleneck.

Least Connections, on the other hand, reacts to the number of active connections. When the slow server remains busy for a longer time, it has more active connections, so Least Connections is more likely to choose the faster or less busy server.

5.4 Extreme Heterogeneity — Larger Difference Between Servers

After that, we made the difference between the two servers larger. The slow server became much slower:

Slow server:

M = 0.50 seconds

V = 1.20 seconds

P = 0.30 seconds

Fast server:

M = 0.10 seconds

V = 0.30 seconds

P = 0.05 seconds

The results were:

Average IP Hash TCT = 266.994 seconds

Average Least Connections TCT = 181.947 seconds

Here the difference between the algorithms became much larger. This was expected because the cost of sending requests to the slow server became much higher.

This experiment showed that server heterogeneity has a strong effect on the result. When the servers are very different, a load balancing algorithm must be careful not to overload the slow server.

5.5 Mapping-Sensitive Scenario - Favorable IP Distribution

After that, we switched the slow and fast servers. This changed the meaning of the IP Hash mapping.

In the previous setup, IP Hash sent more traffic to the slow server, which made IP Hash perform poorly. After switching the server speeds, the same IP mapping became favorable because more traffic was sent to the faster server.

The results were:

Average IP Hash TCT = 158.303 seconds

Average Least Connections TCT = 165.597 seconds

This time, IP Hash was slightly better than Least Connections.

This result was important because it showed that IP Hash can perform well when the IP mapping happens to send more traffic to the faster server. However, this is not because IP Hash understands the system. It is only because the mapping was favorable in this specific case.

5.6 Favorable IP Distribution With Slower Server

We also tested a stronger version of the favorable mapping scenario, where the difference between the fast and slow server was larger.

The results were:

Average IP Hash TCT = 171.697 seconds

Average Least Connections TCT = 181.296 seconds

Again, IP Hash was better because the mapping sent more traffic to the fast server.

This result helped us understand that IP Hash is highly dependent on the client IP distribution. It can be very bad if the mapping sends traffic to the slow server, but it can also be good if the mapping sends more traffic to the fast server.

5.7 Gap Experiments

At this point, we noticed that in some cases there was not a big difference between Least Connections and IP Hash. We wanted to understand whether the request gap was the reason.

We changed the gap values and tested how the algorithms behaved under different request arrival rates.

When the gap is small, many requests arrive close together, so the system is under high load. When the gap is larger, requests are more spread out, so servers have more time to finish old requests before new ones arrive.

The gap experiments showed that the gap does affect the total completion time, but it was not the only explanation for the behavior we saw. The server speeds, and the tie-breaking behavior of Least Connections were also very important.

5.8 Important Observations During the Experiments

During the experiments, we noticed two important points that affected how we interpreted the results.

Observation 1: Failed Packets

Some packets failed during the experiments. This affected the stability of the results and made it harder to compare the algorithms using only one run.

To reduce the number of failed packets, we lowered the number of requests sent during each run. This made the experiments more stable and allowed us to compare the algorithms more reliably.

Because of this, we had to be careful when interpreting the results. We focused on repeated runs and average behavior, not only on a single experiment run.

Observation 2: Tie-Breaking Policy in Least Connections

The second observation was related to the tie-breaking policy in the Least Connections implementation.

When both servers had the same number of active connections, the algorithm chose the first server in the code. This was not a bug and it was a valid implementation of Least Connections. However, it created a deterministic preference toward one server whenever there was a tie.

In a symmetric setup, this preference is usually not very important because both servers have the same processing speed. However, when the servers are not identical, the tie-breaking policy can affect performance. If the preferred server is slower, choosing it during ties may increase the total completion time. If the preferred server is faster, choosing it during ties may improve the result.

This led us to test a different valid implementation of Least Connections, which we called Least Connections - Second Tie. In this version, the algorithm still chooses the server with the fewest active connections, but when there is a tie, it chooses the second server instead of the first one.

This helped us understand that tie-breaking is not an implementation error, but an important design choice that can influence the performance of the algorithm.

5.9 Least Connections — Second Tie

To test the effect of tie-breaking, we added a new algorithm called **Least Connections — Second Tie**.

This algorithm is the same as Least Connections, except for one case: when there is a tie in the number of active connections, it chooses the second server.

The final results showed that this change had a very strong effect.

For example, when **h4 was the fast server** and the gap was 0, the results were:

IP Hash average TCT = 138.14 seconds

Least Connections average TCT = 158.57 seconds

Least Connections — Second Tie average TCT = 81.07 seconds

The request distribution was:

IP Hash:

h3 = 192 requests

h4 = 384 requests

Least Connections:

h3 = 223 requests

h4 = 353 requests

Least Connections — Second Tie:

h3 = 81 requests

h4 = 495 requests

Since h4 was the fast server, sending more requests to h4 improved the total completion time significantly.

This result showed that Least Connections alone is not always enough. The tie-breaking rule can make a major difference, especially when the servers have different speeds.

---

6. Main Conclusions

From the experiments, we reached several conclusions:

1. **When the servers are similar, the difference between IP Hash and Least Connections is small.**
   Least Connections may be slightly better, but the improvement is not always large.
2. **When one server is slower, request distribution becomes much more important.**
   Sending too many requests to the slow server increases the total completion time.
3. **Least Connections usually works better in heterogeneous setups.**
   It reacts to active connections and avoids overloaded servers better than IP Hash.
4. **Tie-breaking matters.**
   In Least Connections, when both servers have the same number of active connections, the choice of server can strongly affect performance.
5. **Least Connections — Second Tie gave the best result in the final scenario.**
   This happened because it sent more traffic to the fast server when there was a tie.

---

7. Final Summary

The project started with a simple comparison between IP Hash and Least Connections. At first, we expected Least Connections to clearly outperform IP Hash. However, the experiments showed that the behavior of load balancing algorithms depends on many details.

In symmetric servers, the difference between the algorithms was small. In heterogeneous servers, Least Connections usually performed better because it reacted to the current load. However, IP Hash sometimes performed well when the client IP mapping happened to favor the fast server.

The most important insight was that implementation details matter. The original Least Connections algorithm always chose the first server during a tie, which affected the results. After adding Least Connections — Second Tie, we saw a significant improvement in the final scenario.

Therefore, the project showed that load balancing performance is not determined only by the algorithm name. It also depends on server speeds, traffic patterns, client mapping, request timing, failed packets, and tie-breaking rules.

---

8. References

- Load Balancer — GeeksforGeeks (https://www.geeksforgeeks.org/system-design/what-is-load-balancer-system-design/)
- Load Balancing Algorithms — GeeksforGeeks (https://www.geeksforgeeks.org/system-design/load-balancing-algorithms/)
- What is IP Hash? — Web Hosting Geeks Blog (https://webhostinggeeks.com/blog/what-is-ip-hash/)
- loadBalancer/loadBalancerLab.py at master · cs236341/loadBalancer · GitHub